

PowerShell Workshop

Session 1 — Foundations: Functions, Variables, Parameters

```
PS C:\workshop> _
```

What We'll Cover

Our roadmap for the session.

- **Getting oriented** — Windows PowerShell vs. PowerShell
 - **Working with data** — variables, data types, strings
 - **Functions** — what they are, and naming (Verb-Noun + prefix)
 - **Parameters** — feeding data into functions
 - **Build time** — hands-on exercise
-

Windows PowerShell vs. PowerShell

Windows PowerShell

v1.0 – v5.1

- Legacy .NET Framework
- Windows-only
- No longer developed

PowerShell

v6.0+ (formerly Core)

- Modern, open-source .NET
- Cross-platform
- The active standard

Variables

Named placeholders that store data.

- Start with \$
- Assign with =
- Can change anytime

```
PS> $serverName = "Production-01"  
PS> Write-Host $serverName  
Production-01
```

```
PS> $serverName = "Backup-02"  
PS> Write-Host $serverName  
Backup-02
```

```
# Tip: capture command output in a variable – $services = Get-Service
```

Common Data Types

Type your inputs — PowerShell rejects the wrong kind.

[string]

Text & characters

[int]

Whole numbers

[bool]

True / false flags

[datetime]

Dates & times

```
[int]$year = 2024    # passing "hello" here is rejected automatically
```

Handling Strings

String Interpolation

Recommended

```
Write-Host "Hello, $name!"
```

String Formatting (-f)

Useful for many variables

```
Write-Host ("Hello, {0}!" -f $name)
```

String Concatenation

Avoid

```
Write-Host ("Hello, " + $name + "!" )
```

What Are Functions?

Define once, call by name.

Reusability

Write once, use everywhere.

Maintainability

Fix in one place.

Readability

Cleaner, logical structure.

```
function Get-Greeting {  
    Write-Host "Hello from a function!"  
}
```

Naming: Verb-Noun + Prefix

Prefix your functions so they never clash with built-in cmdlets.

The Verb-Noun Standard

Verb-Noun pattern, e.g. Get-Service.

Custom Prefixing

Add your organization or project initials to the noun.

```
# Risky – could shadow a built-in cmdlet  
Get-Date
```

```
# Safe – prefixed with the organization (INF = Infraworkz)  
Get-INFDate
```


Parameters

Inputs that make functions dynamic.

- Defined in param()
- Mandatory or optional
- Optional ones take defaults

```
function Get-INFGreeting {  
    param(  
        [Parameter(Mandatory=$true)]  
        [string]$name,  
  
        [string]$greeting = "Hello"  
    )  
    Write-Host "$greeting, $name!"  
}
```

Tip: a Mandatory parameter prompts the caller automatically if it's left out

Your Turn — Let's Build

- Open the repo below — the README on the page walks you through it
- Work through the exercises at your own pace

Exercises live here:



github.com/Danielvdmeer/powershell_workshop

```
PS C:\workshop> .\start-exercises.ps1
```
